deeplearning.ai

Case Studies

Why look at case studies?

Outline

Classic networks:

- LeNet-5 ←
- AlexNet ←
- VGG ←

ResNet (152)

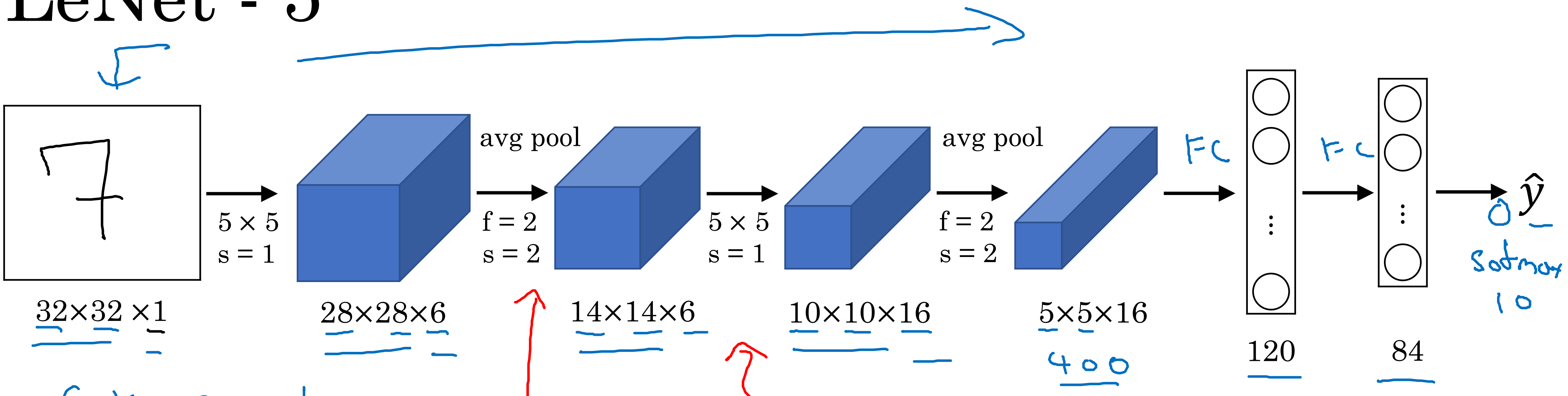
Inception

deeplearning.ai

Case Studies

Classic networks

LeNet - 5



60K parameters.

n_H, n_W ↓ n_C ↑

non-linearity after pooling

$n_H \times n_W \times n_C$ $f \times f \times n_C$

conv pool conv pool fc fc output

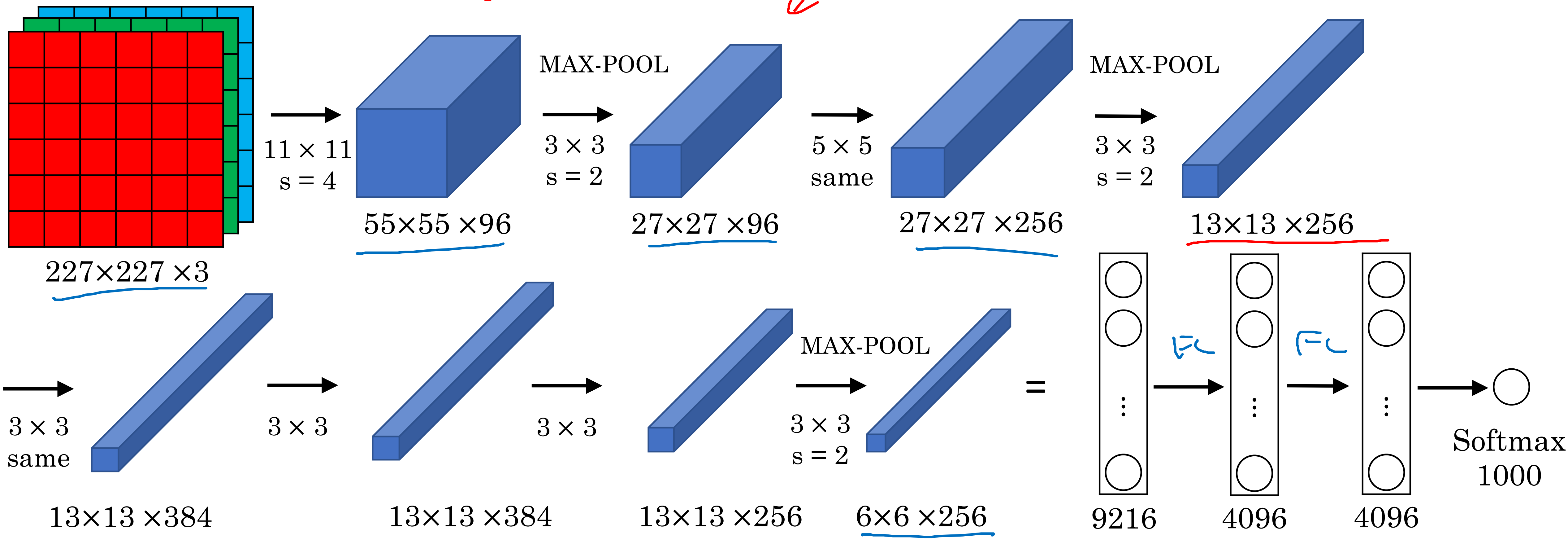
Advanced: sigmoid/tanh ReLU

II III.

[LeCun et al., 1998. Gradient-based learning applied to document recognition]

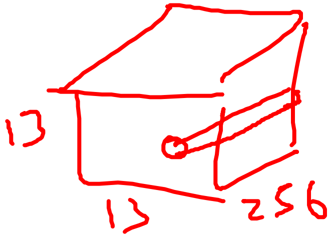
Andrew Ng

AlexNet



9216

- Similar to LeNet, but much bigger.   ~60M parameters
- ReLU
- Multiple GPUs.
- Local Response Normalization (LRN)



[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

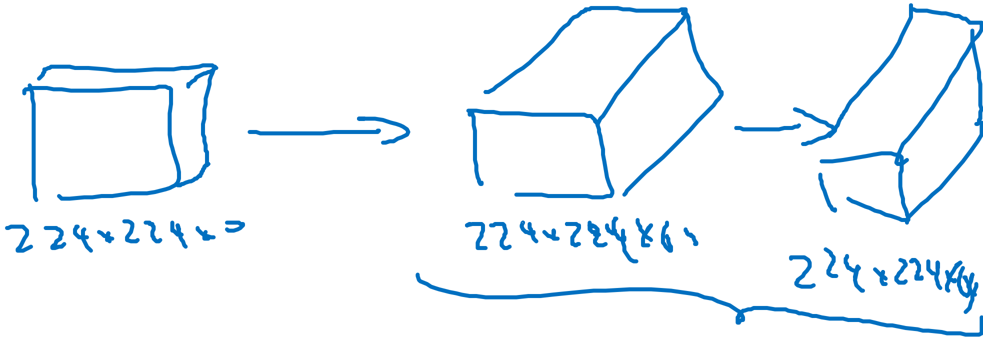
Andrew Ng

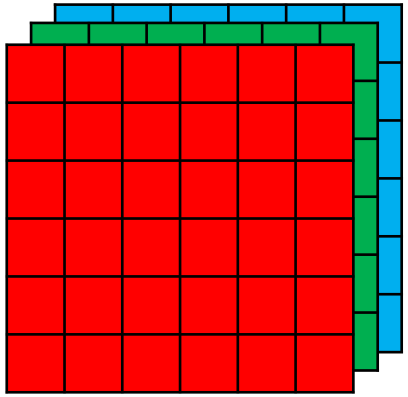
VGG - 16

VGG-19

CONV = 3×3 filter, s = 1, same

MAX-POOL = 2×2 , s = 2





224×224 ×3 → [CONV 64] ×2 → 224×224×64 → POOL → 112×112 ×64 → [CONV 128] ×2 → 112×112 ×128 → POOL → 56×56 ×128

→ [CONV 256] ×3 → 56×56 ×256 → POOL → 28×28 ×256 → [CONV 512] ×3 → 28×28 ×512 → POOL → 14×14×512

→ [CONV 512] ×3 → 14×14 ×512 → POOL → 7×7×512 → FC 4096 → FC 4096 → Softmax 1000

$n_H, n_W \downarrow$ $n_c \uparrow$

~138M

[Simonyan & Zisserman 2015. Very deep convolutional networks for large-scale image recognition]

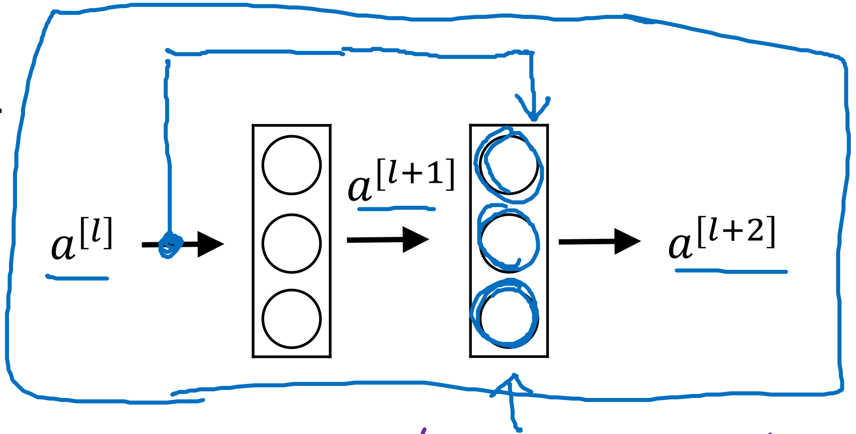
Andrew Ng

deeplearning.ai

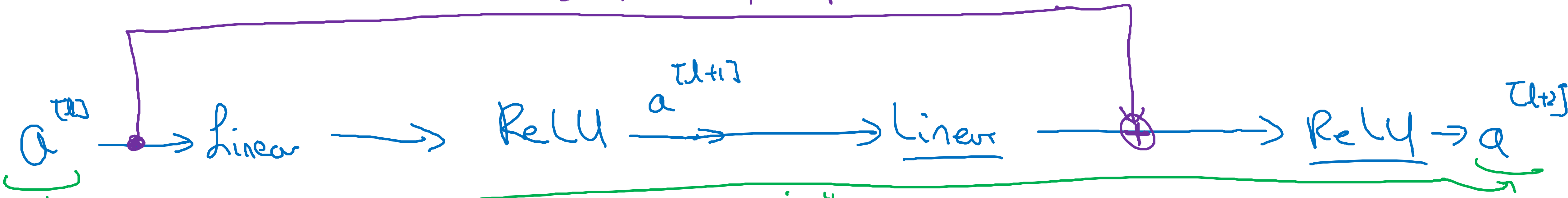
Case Studies

Residual Networks (ResNets)

Residual block



"short cut"/skip connection

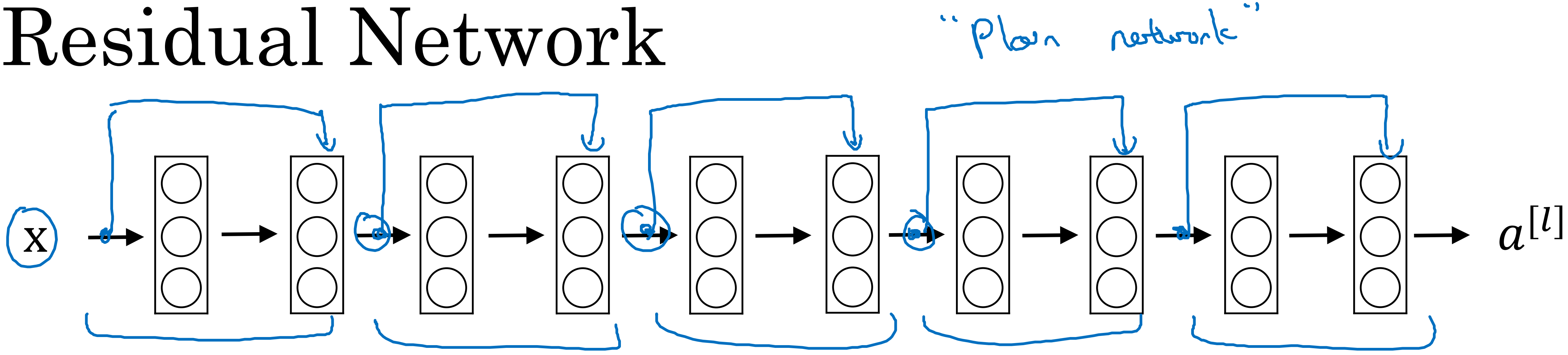


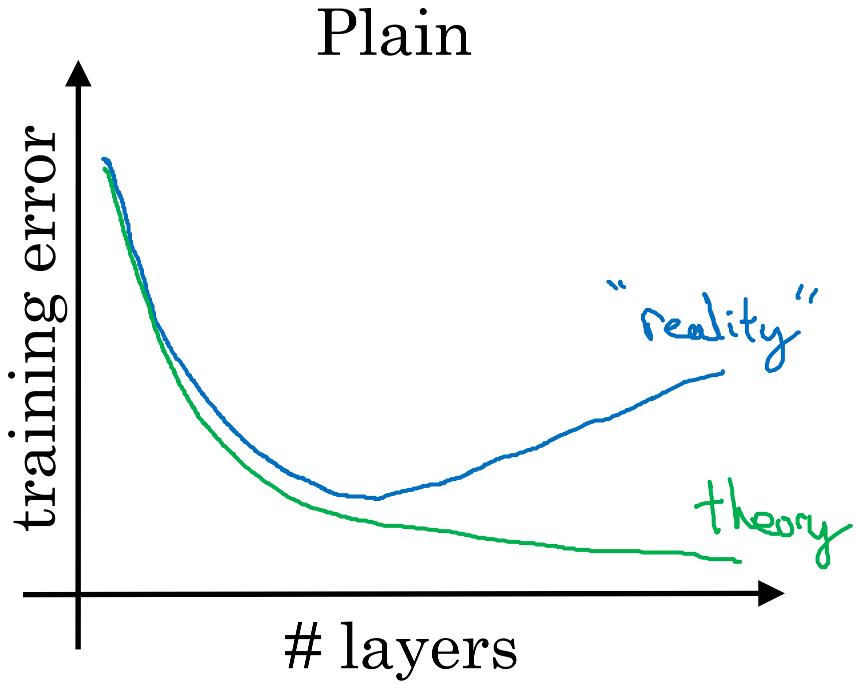
$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$ $\quad a^{[l+1]} = g(z^{[l+1]})$ $\quad z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$ $\quad \boxed{\cancel{a^{[l+2]} = g(z^{[l+2]})}}$

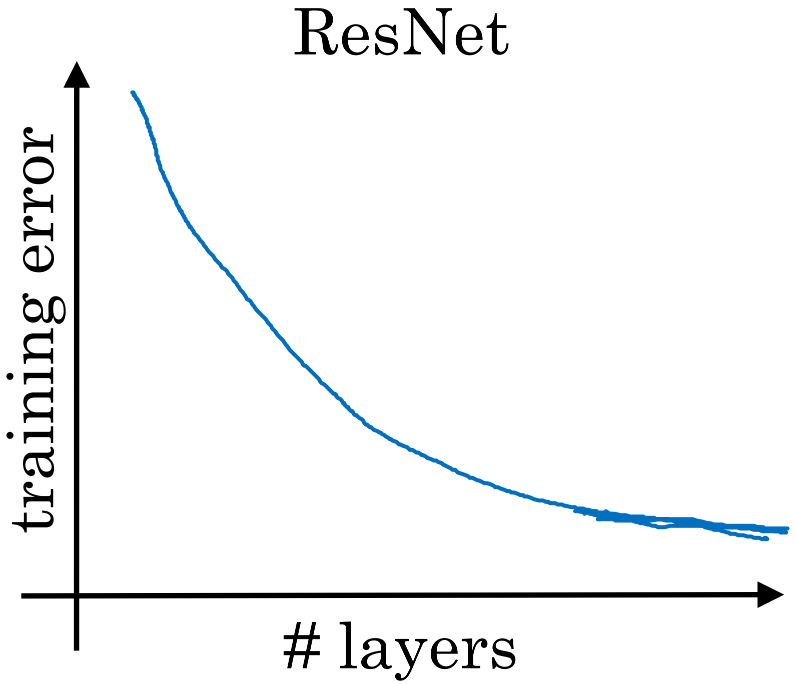
$$a^{[l+2]} = g\left(z^{[l+2]} + a^{[l]}\right)$$

[He et al., 2015. Deep residual networks for image recognition]

Residual Network







[He et al., 2015. Deep residual networks for image recognition]

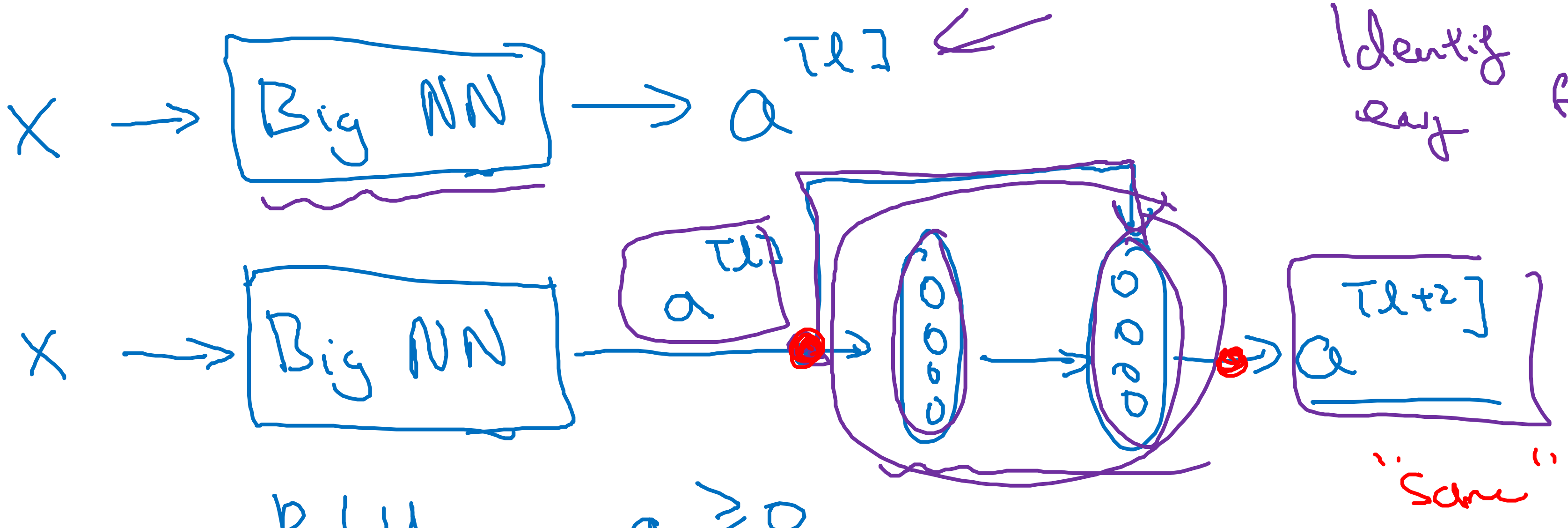
Andrew Ng

deeplearning.ai

Case Studies

Why ResNets work

Why do residual networks work?



Identity function is easy for Residual block to learn!

ReLU. $a \geq 0$

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

$$= g(\cancel{W^{[l+2]} a^{[l+1]} + b^{[l+2]}} + W_s a^{[l]}) = g(a^{[l]})$$

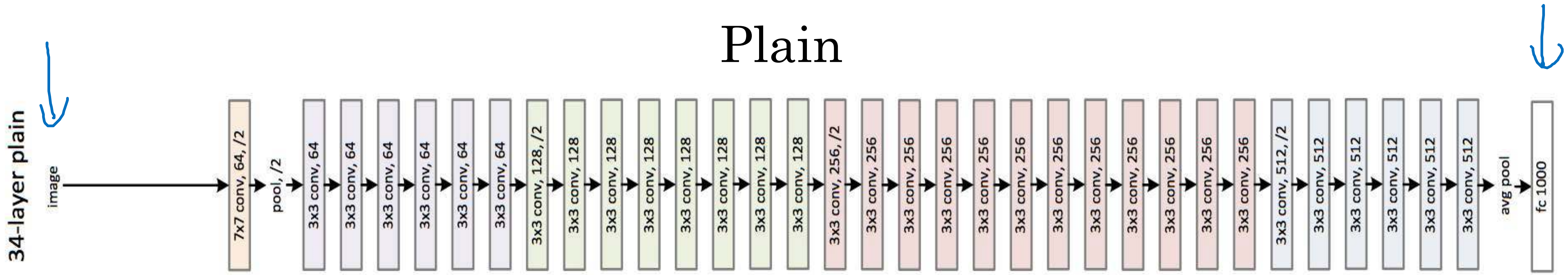
$$= a^{[l]}$$

256    $W_s \in \mathbb{R}^{256 \times 128}$ 128

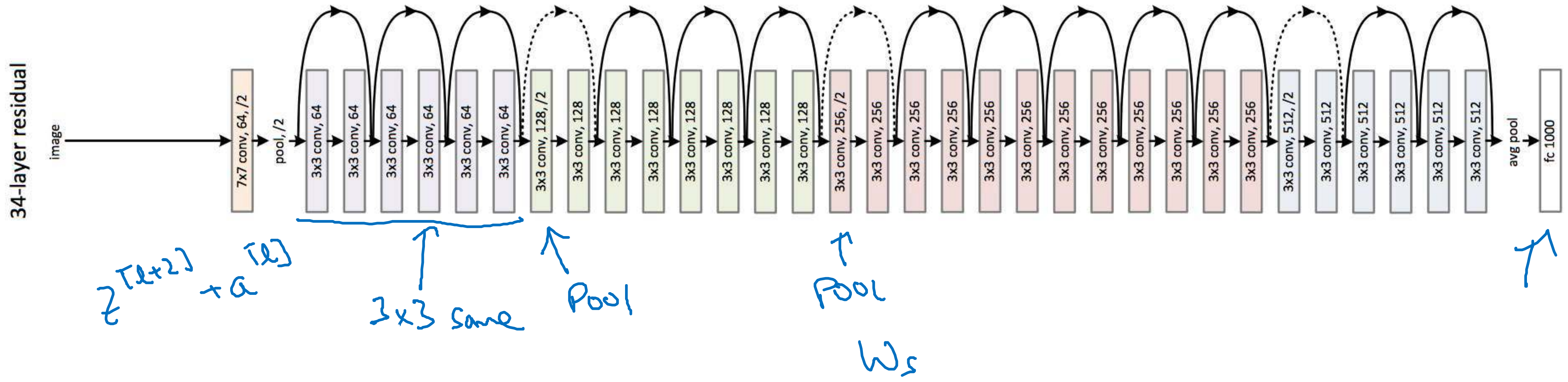
If $W^{[l+2]} = 0$, $b^{[l+2]} = 0$

ResNet

Plain



ResNet



[He et al., 2015. Deep residual networks for image recognition]

Andrew Ng

deeplearning.ai

Case Studies

Network in Network and 1×1 convolutions

Why does a 1 × 1 convolution do?

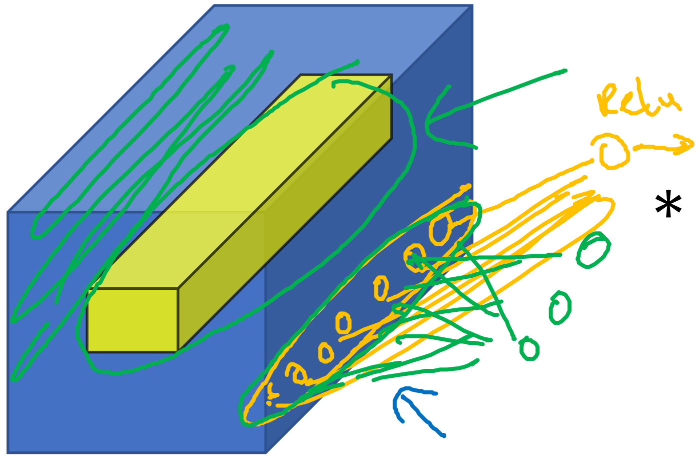
1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

6 × 6 × 1

$*$ $\boxed{2}$ =

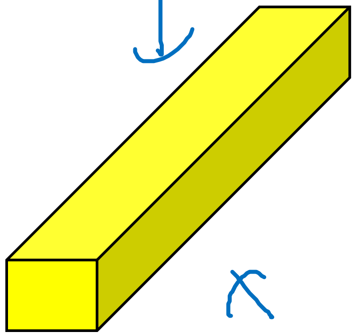
2	4	6	...		

32 → # filters. $n_c^{[l+1]}$



6 × 6 × 32

*

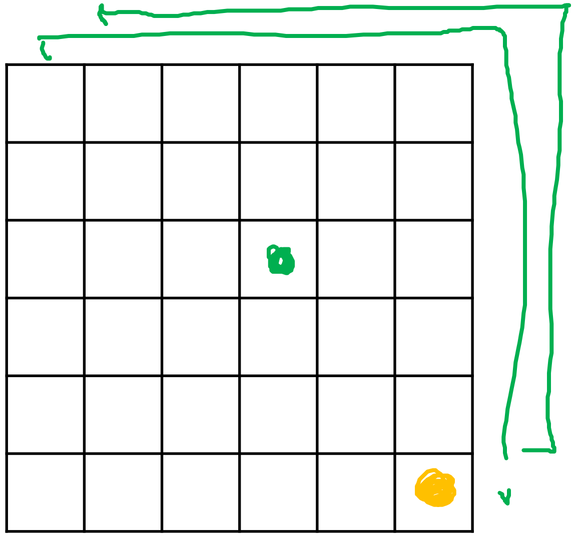


1 × 1 × 32

=

ReLU

Network in Network

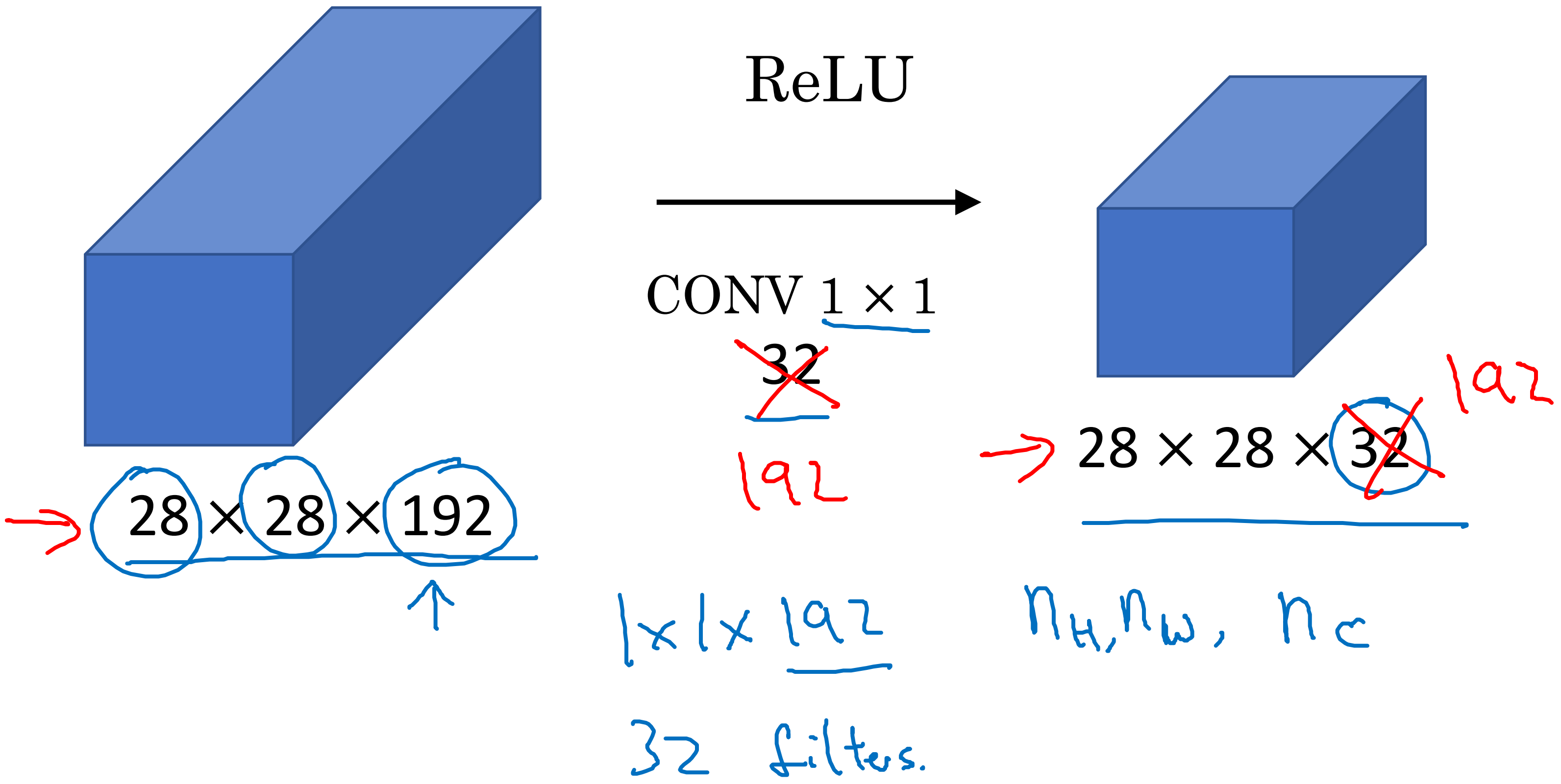


6 × 6 × # filters

[Lin et al., 2013. Network in network]

Andrew Ng

Using 1×1 convolutions



[Lin et al., 2013. Network in network]

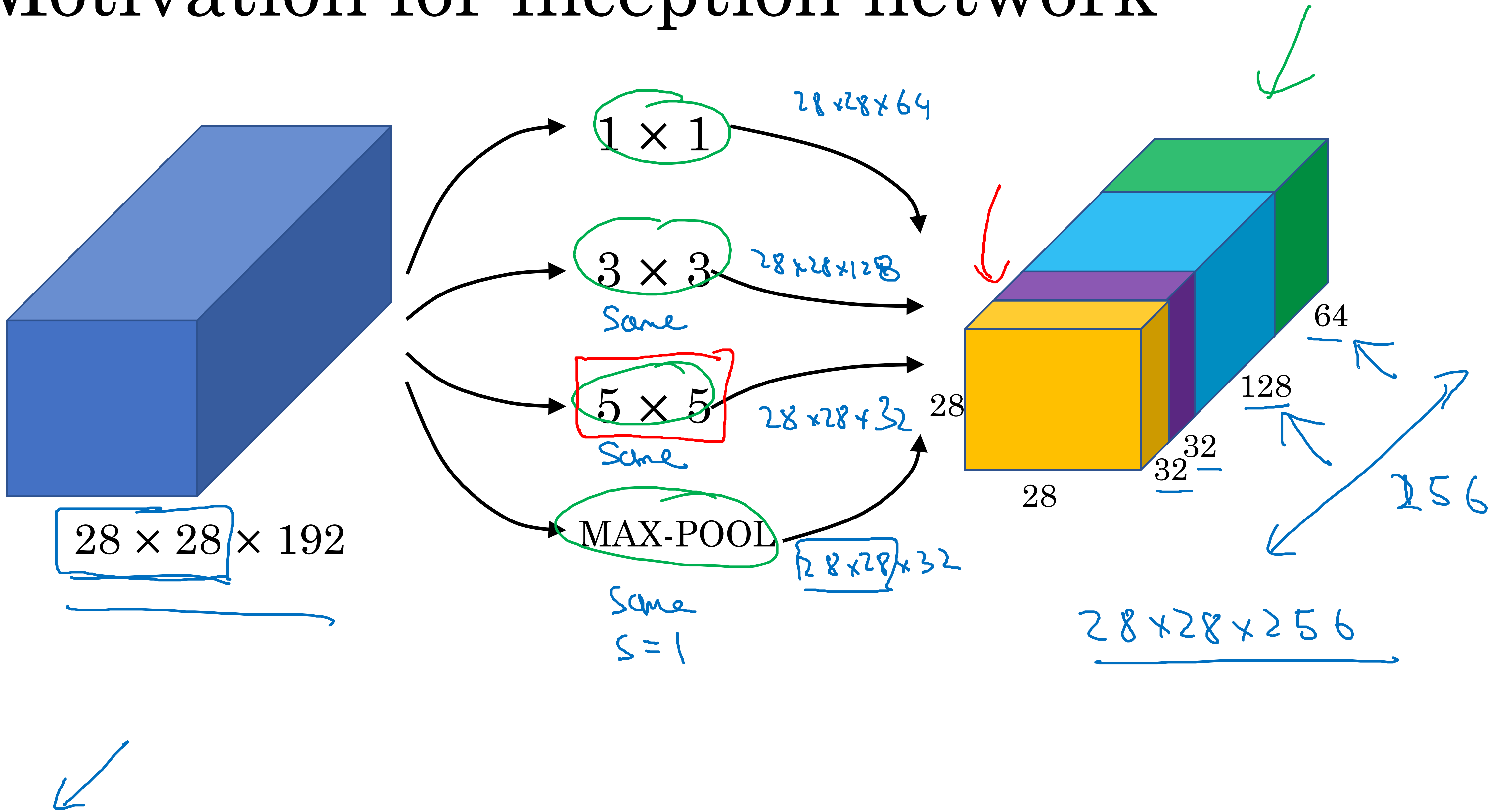
Andrew Ng

deeplearning.ai

Case Studies

Inception network motivation

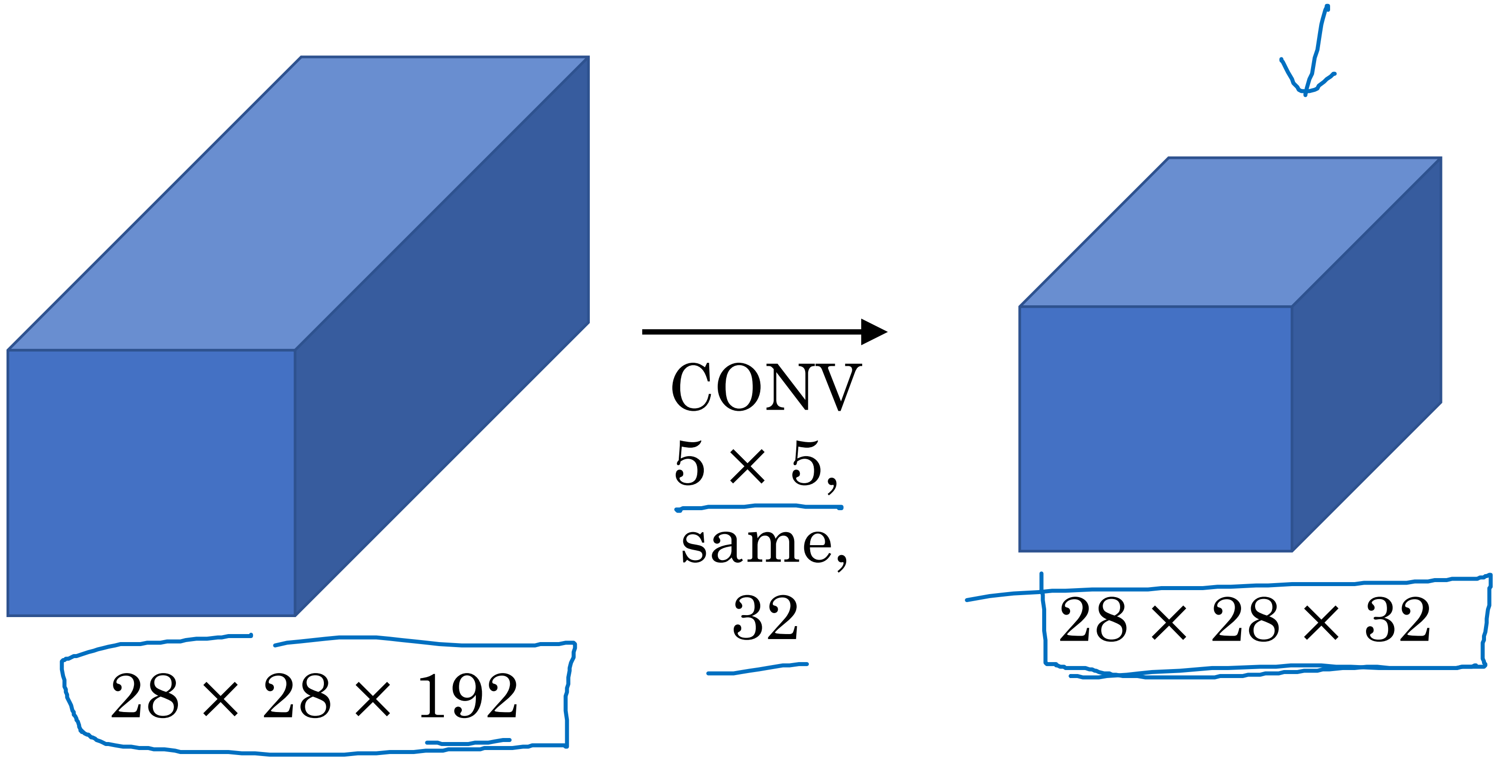
Motivation for inception network



[Szegedy et al. 2014. Going deeper with convolutions]

Andrew Ng

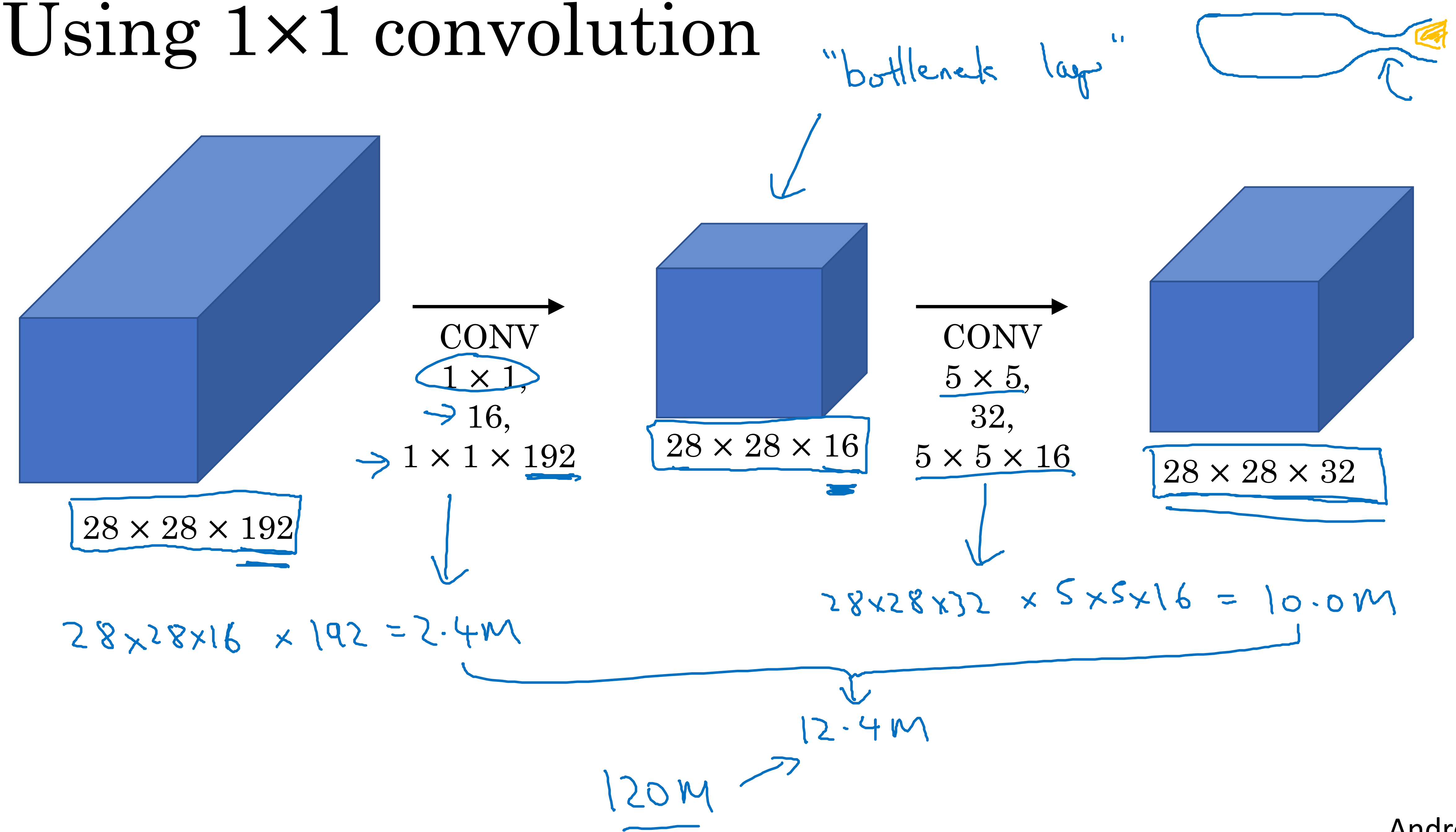
The problem of computational cost



32 filters. filters are 5×5×192.

$28 \times 28 \times 32 \times 5 \times 5 \times 192 = 120M$.

Using 1×1 convolution

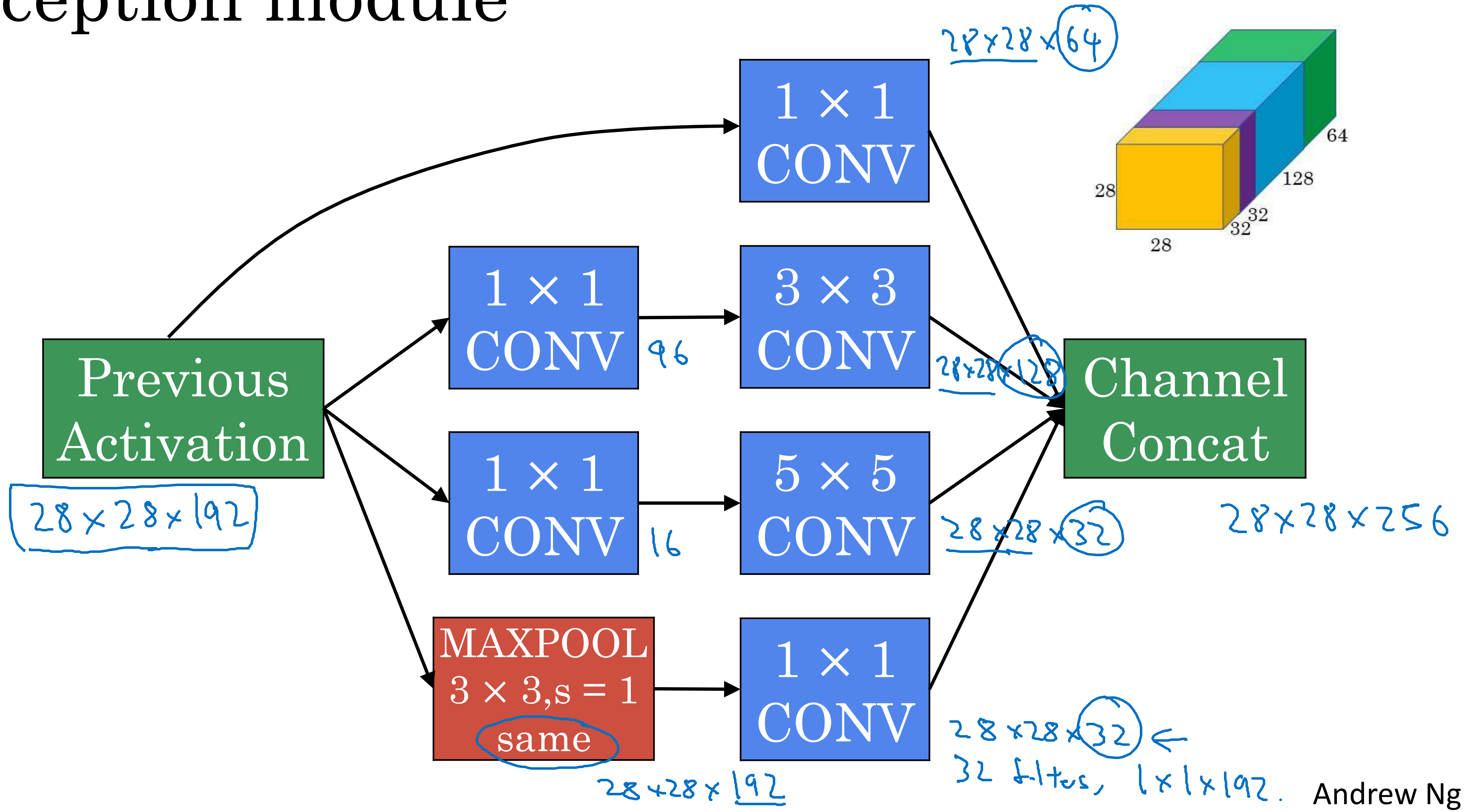


deeplearning.ai

Case Studies

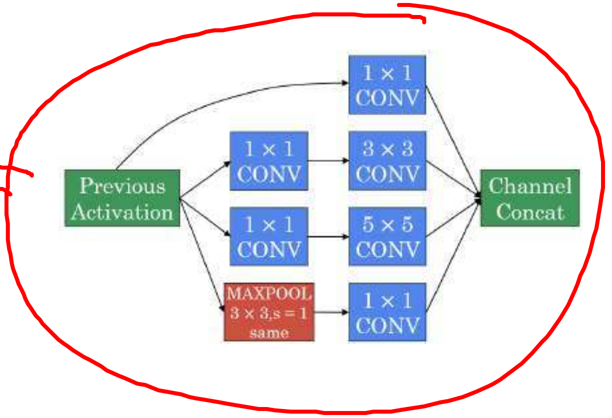
Inception network

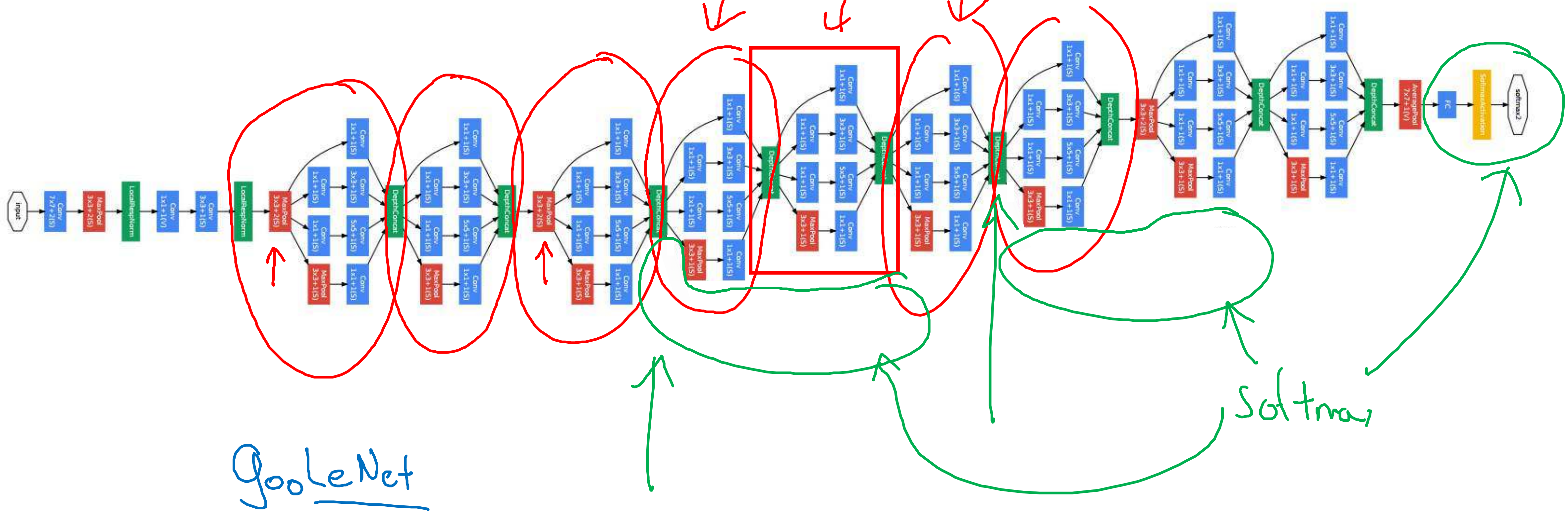
Inception module



Andrew Ng

Inception network





[Szegedy et al., 2014, Going Deeper with Convolutions]

http://knowyourmeme.com/memes/we-need-to-go-deeper

deeplearning.ai

Convolutional Neural Networks

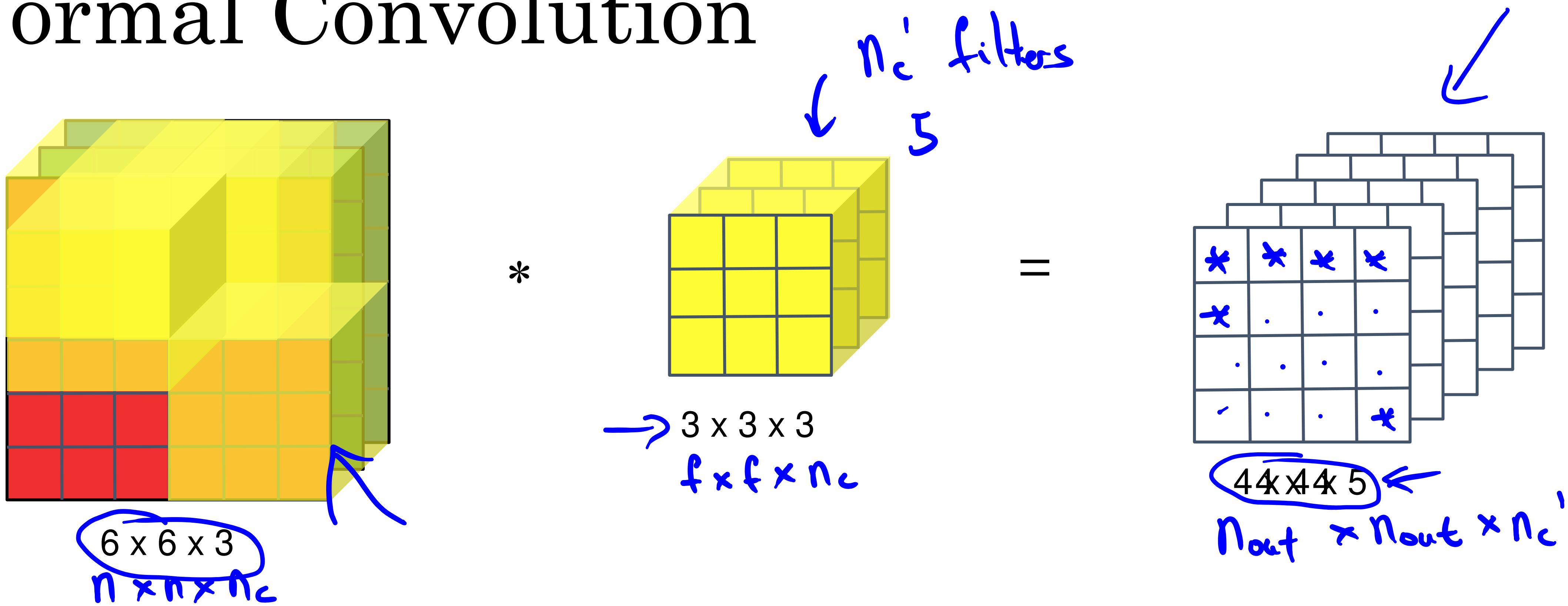
MobileNet

Motivation for MobileNets

- Low computational cost at deployment
- Useful for mobile and embedded vision applications
- Key idea: Normal vs. depthwise-separable convolutions

[Howard et al. 2017, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications]

Normal Convolution

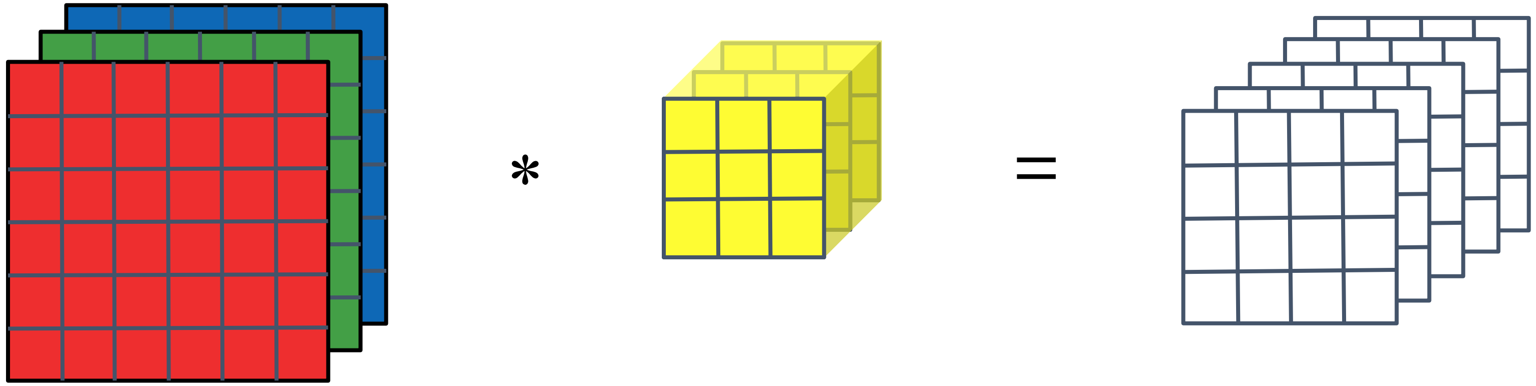


Computational cost = #filter params x # filter positions x # of filters

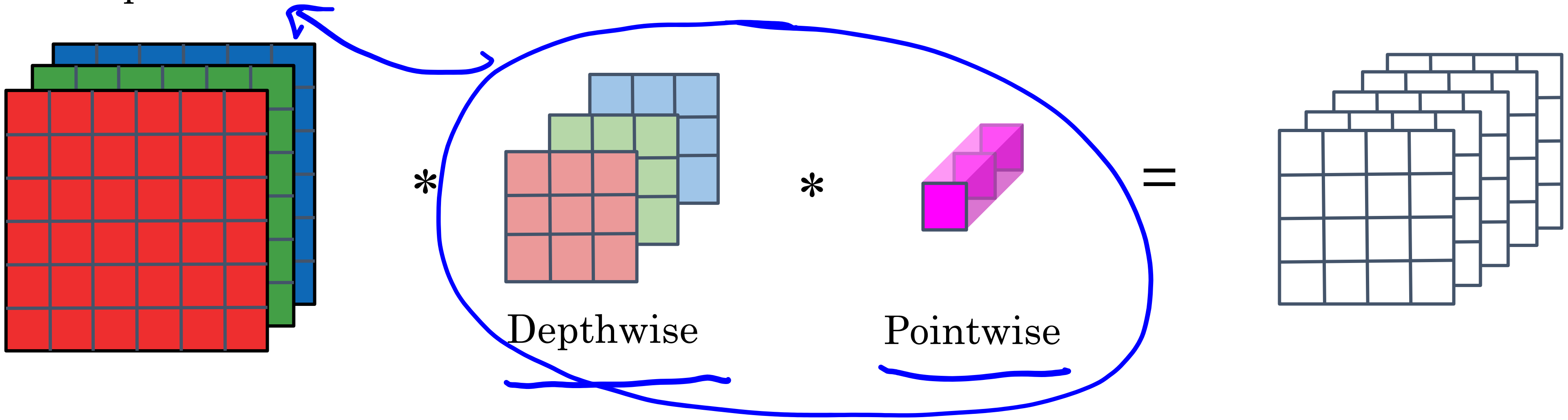
$2160 = 3 \times 3 \times 3 \times 4 \times 4 \times 5$

Depthwise Separable Convolution

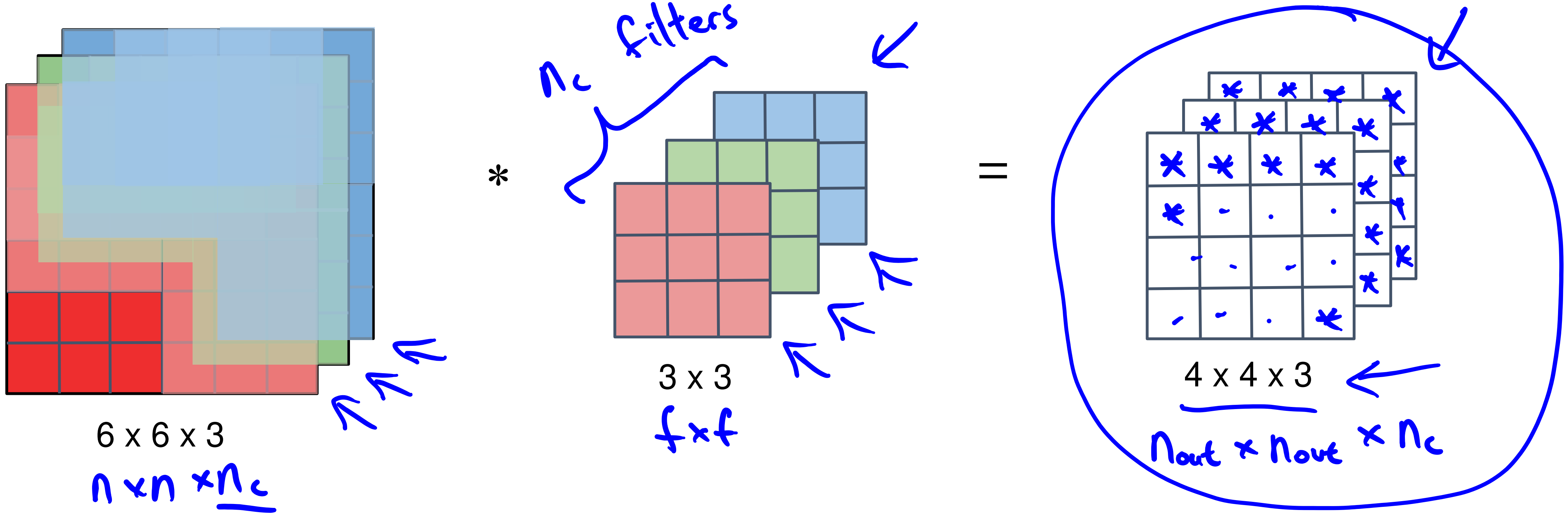
Normal Convolution



Depthwise Separable Convolution



Depthwise Convolution



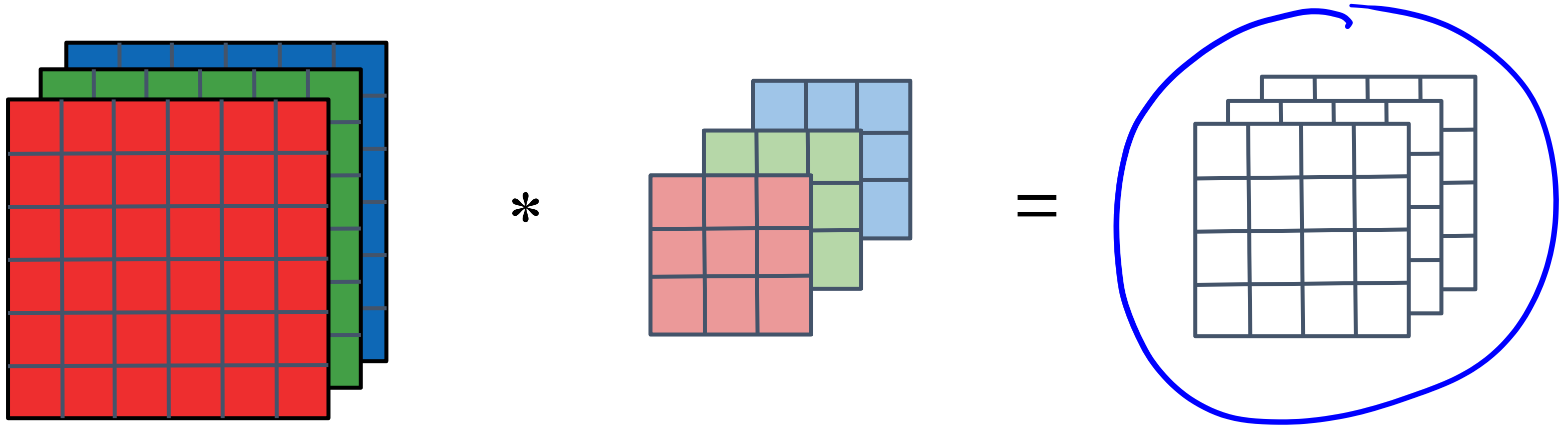
Computational cost = #filter params x # filter positions x # of filters

$432 = \underbrace{3 \times 3}_{} \times \underbrace{4 \times 4 \times 3}_{}$

Andrew Ng

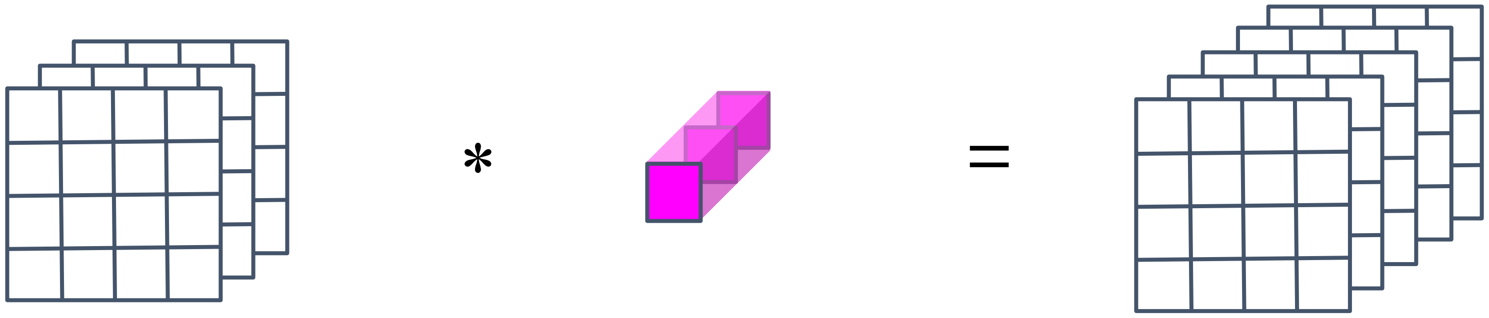
Depthwise Separable Convolution

Depthwise Convolution



432

Pointwise Convolution

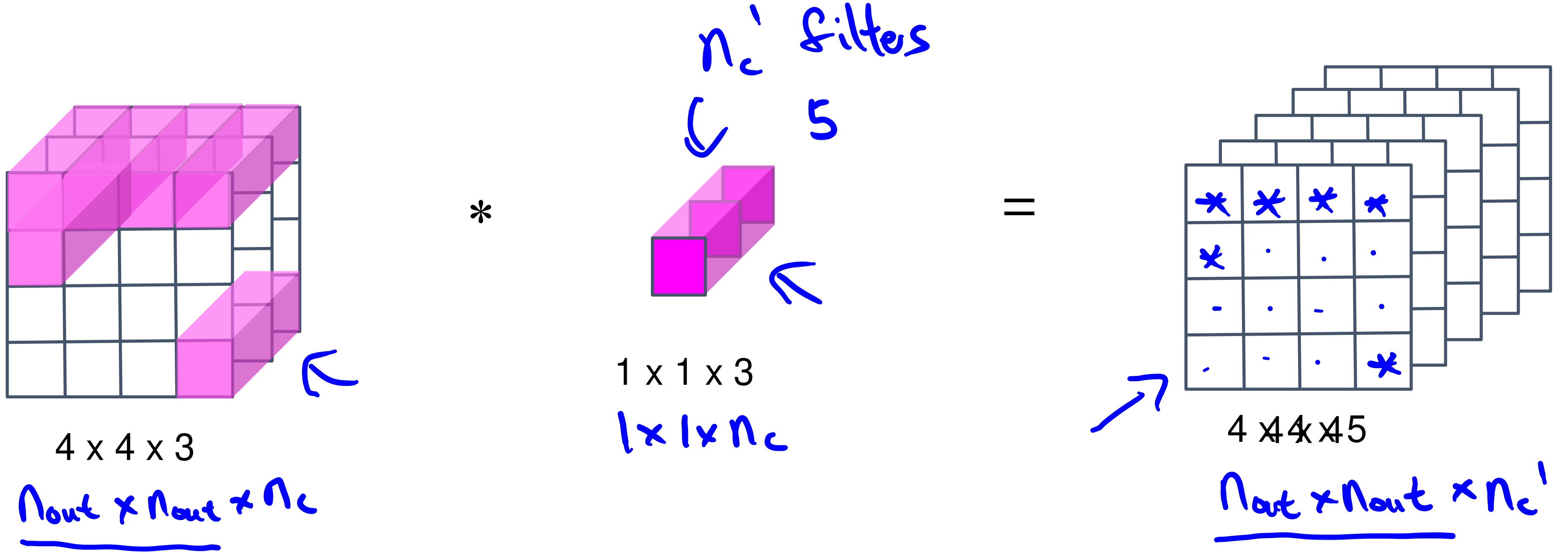


4x4x3

4x4x5

Andrew Ng

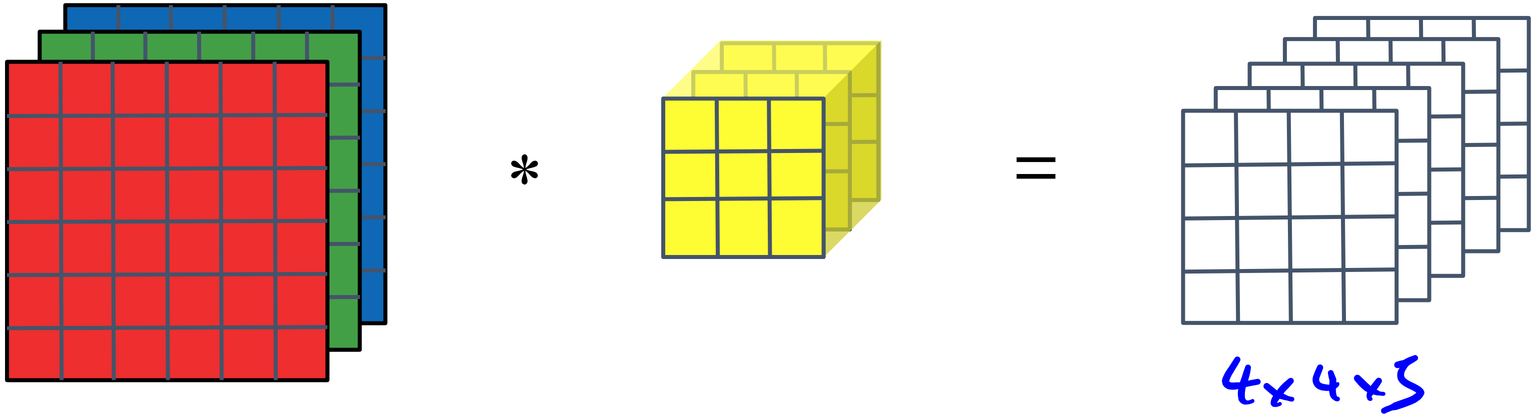
Pointwise Convolution



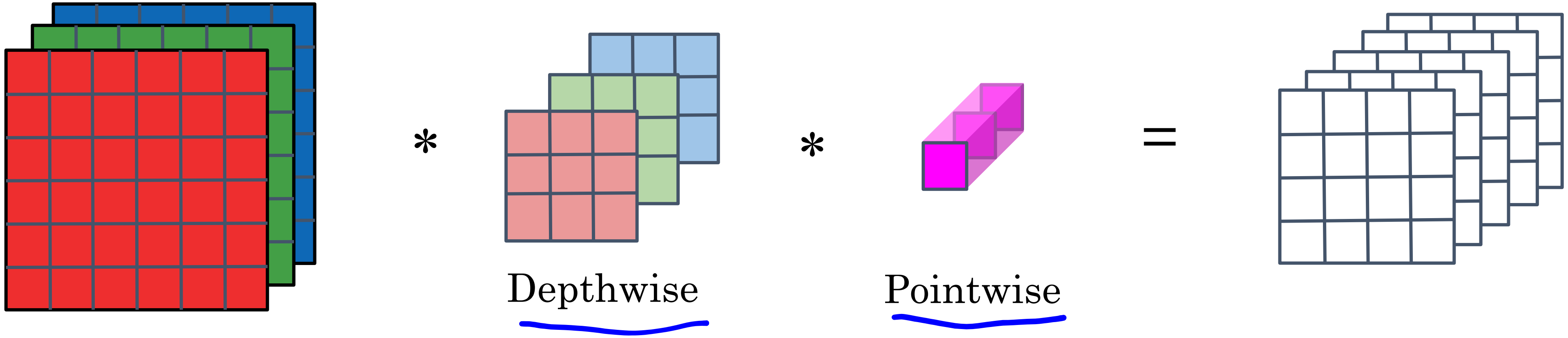
Computational cost	=	#filter params	x	# filter positions	x	# of filters
240	=	1x1x3	x	4x4	x	5

Depthwise Separable Convolution

Normal Convolution



Depthwise Separable Convolution



Andrew Ng

Cost Summary

Cost of normal convolution 2160

Cost of depthwise separable convolution

depthwise + pointwise

432 + 240 = 672

$$\frac{672}{2160} = 0.31$$

$$= \frac{1}{n_c'} + \frac{1}{f^2}$$

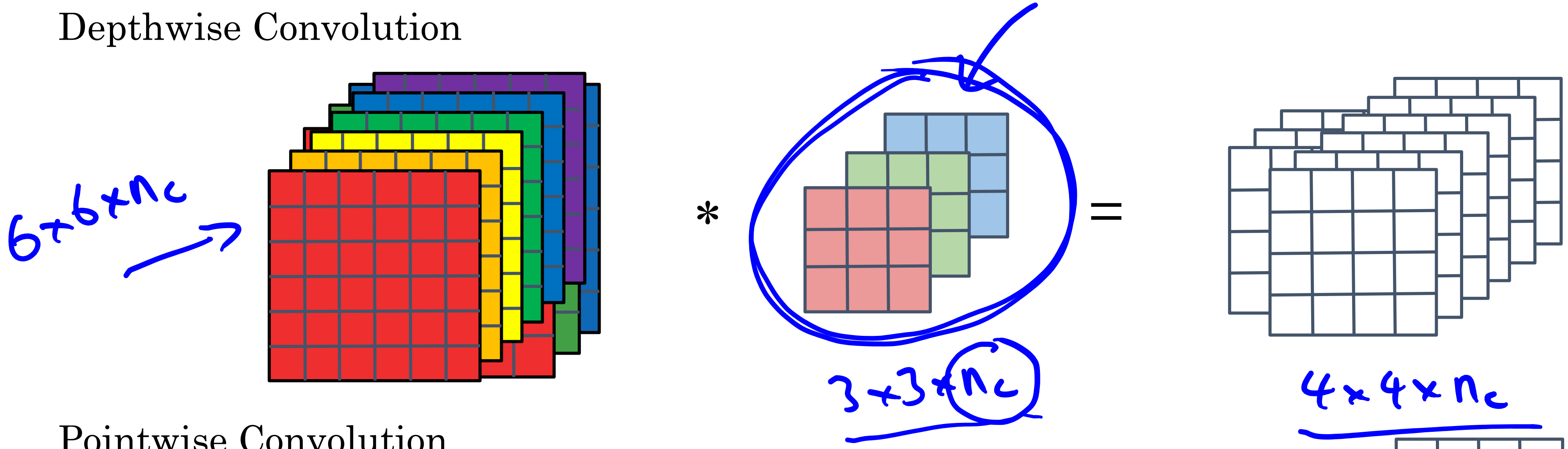
$$\frac{1}{5} + \frac{1}{9}$$

$$= \frac{1}{512} + \frac{1}{3^2}$$

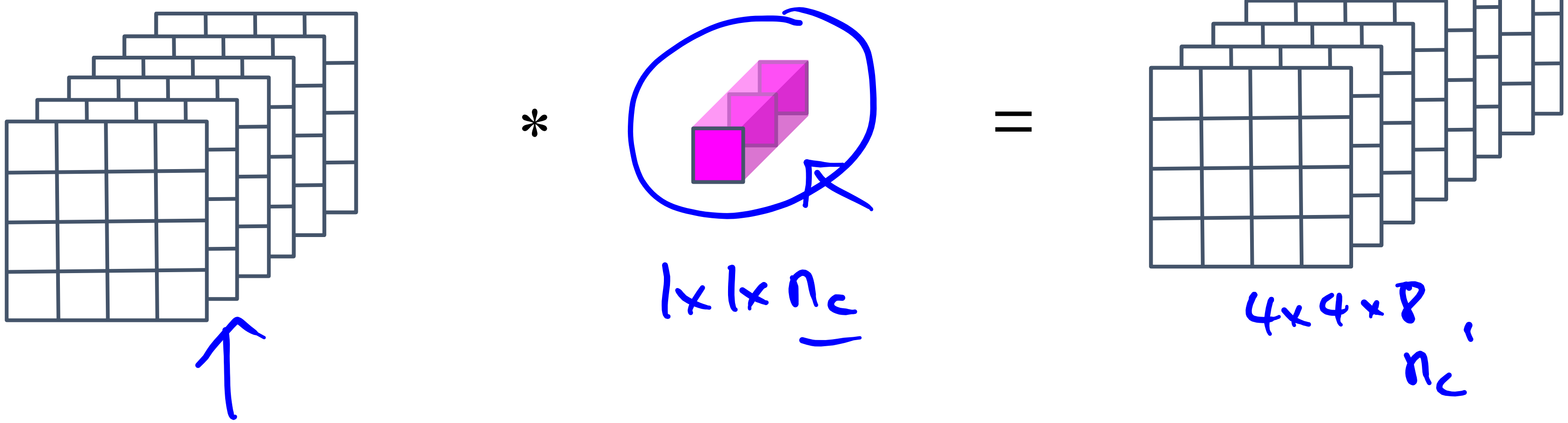
~10 times cheaper

Depthwise Separable Convolution

Depthwise Convolution



Pointwise Convolution



Andrew Ng

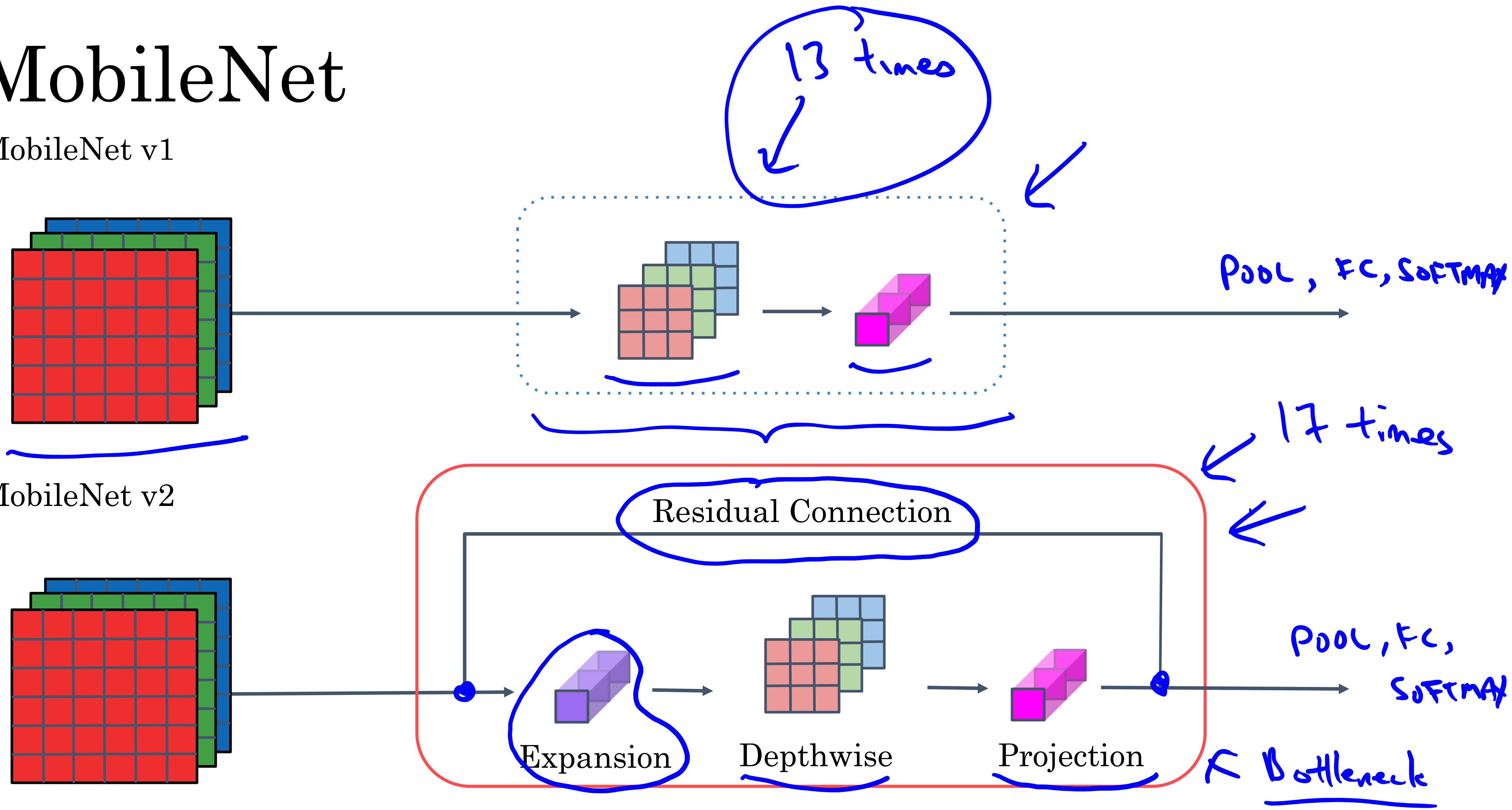
deeplearning.ai

Convolutional Neural Networks

MobileNet Architecture

MobileNet

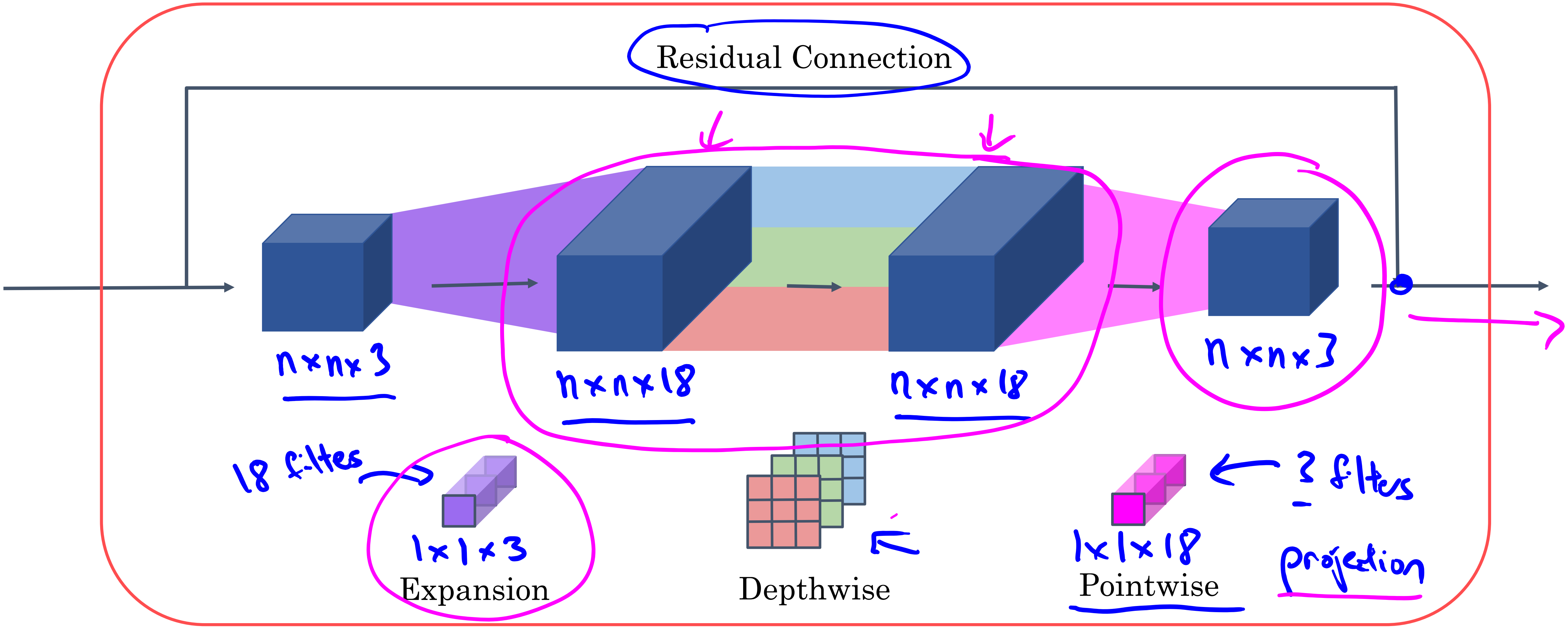
MobileNet v1



[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

Andrew Ng

MobileNet v2 Bottleneck

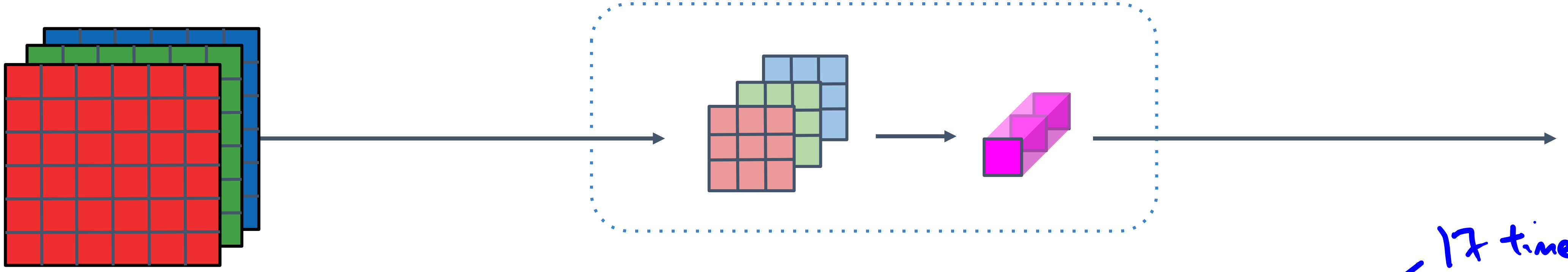


[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

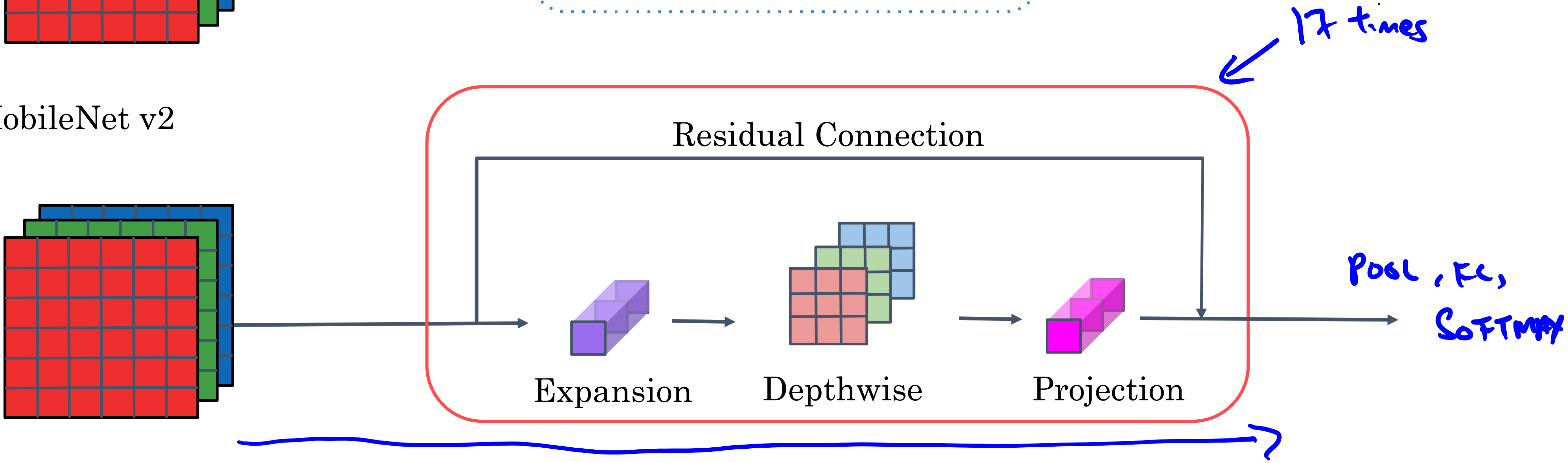
Andrew Ng

MobileNet

MobileNet v1

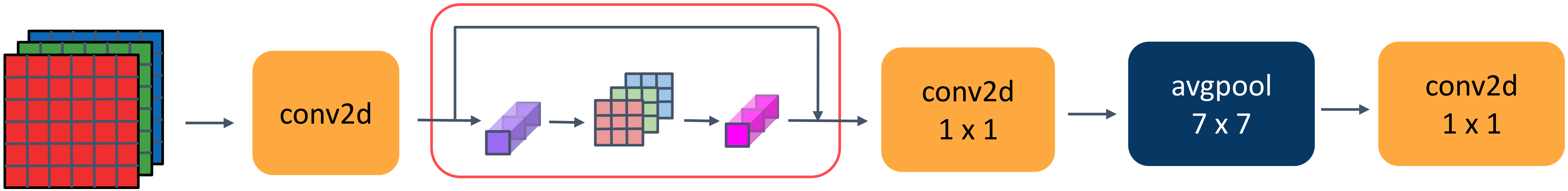


MobileNet v2



[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

MobileNet v2 Full Architecture



[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

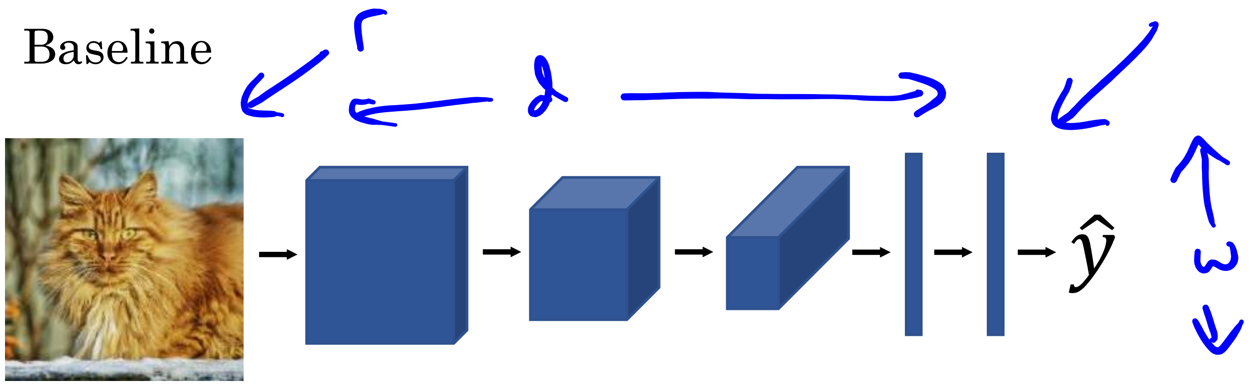
Convolutional Neural Networks

EfficientNet

deeplearning.ai

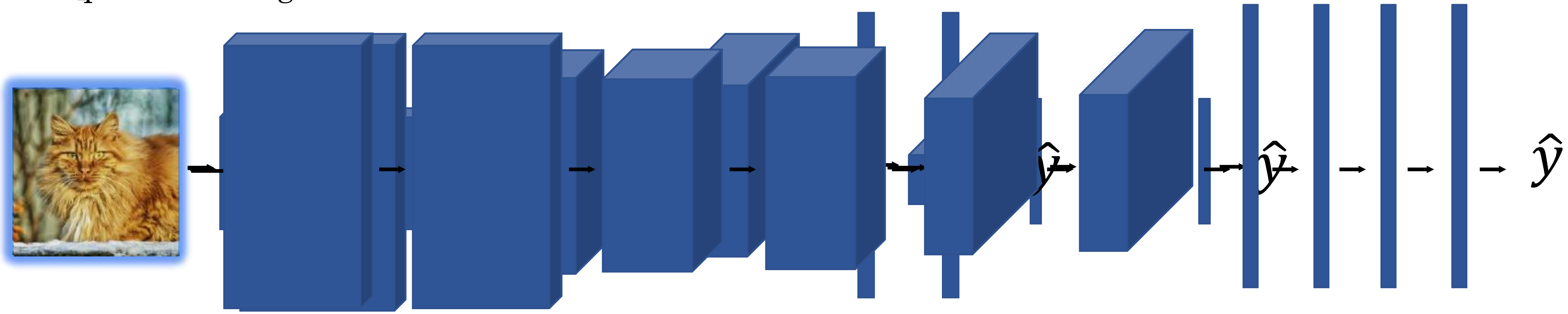
EfficientNet

Baseline



r resolution
d depth
w width

Wider
Deeper
Higher Resolution
Compound scaling



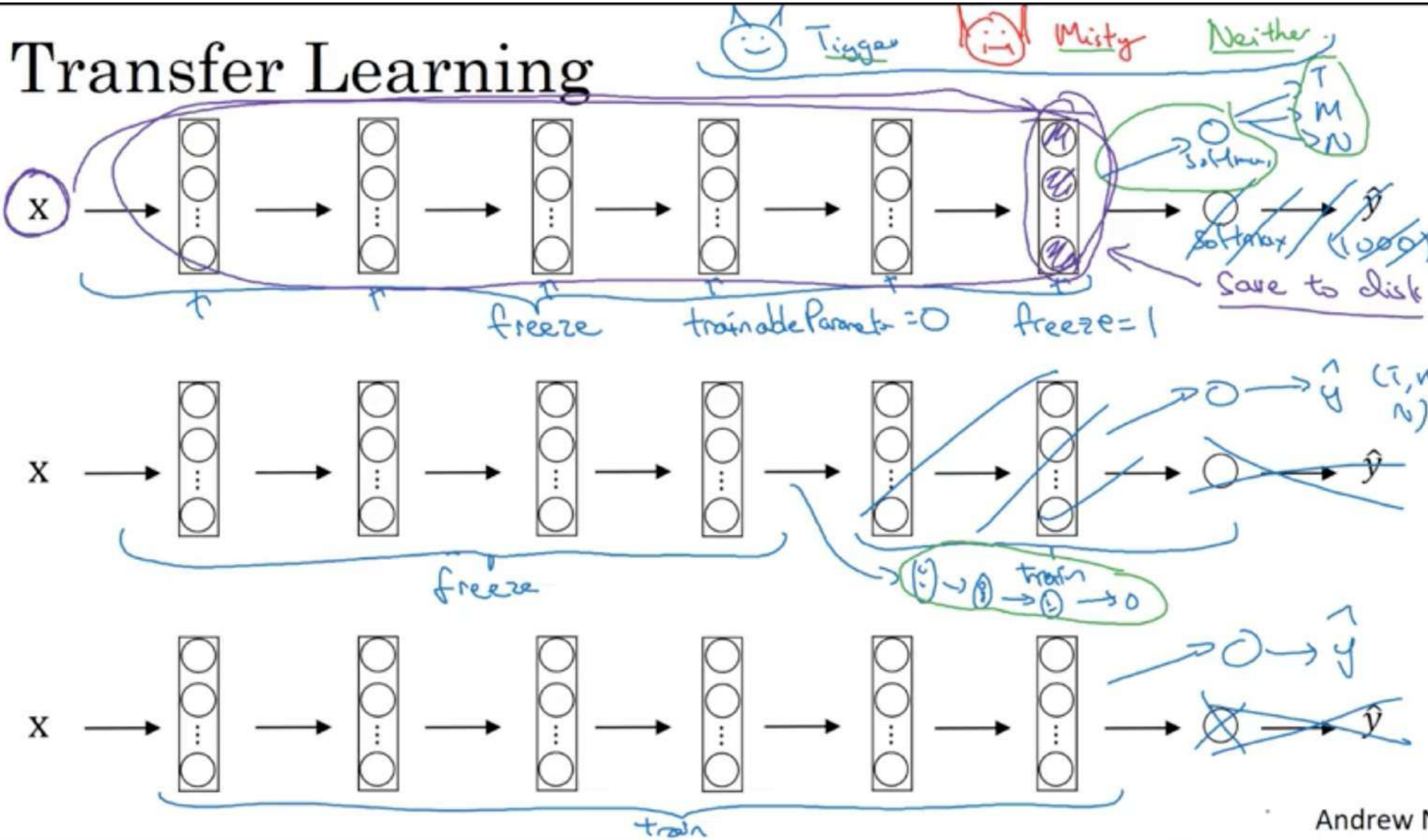
[Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks]

Practical advice for using ConvNets

Transfer Learning

deeplearning.ai

Transfer Learning



Andrew Ng

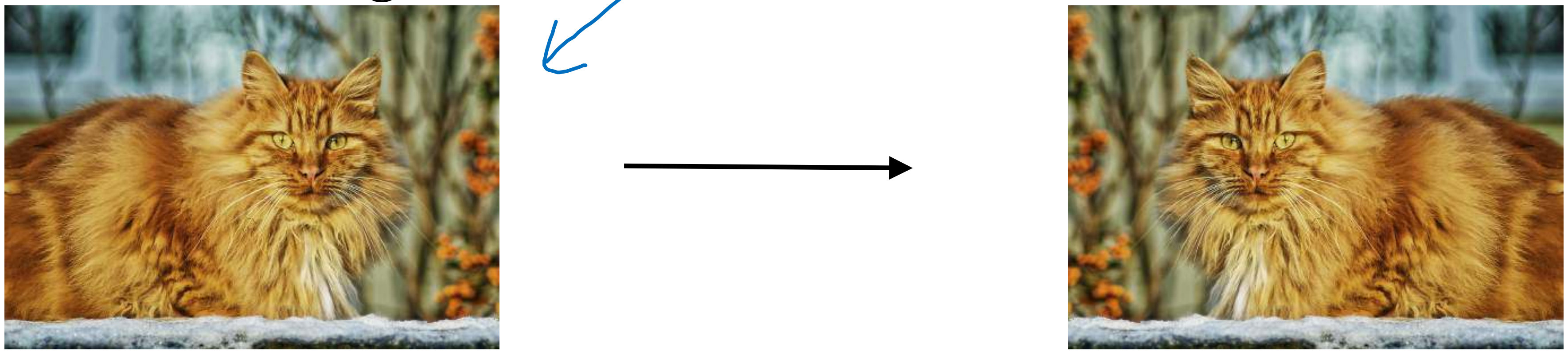
Practical advice for using ConvNets

Data augmentation

deeplearning.ai

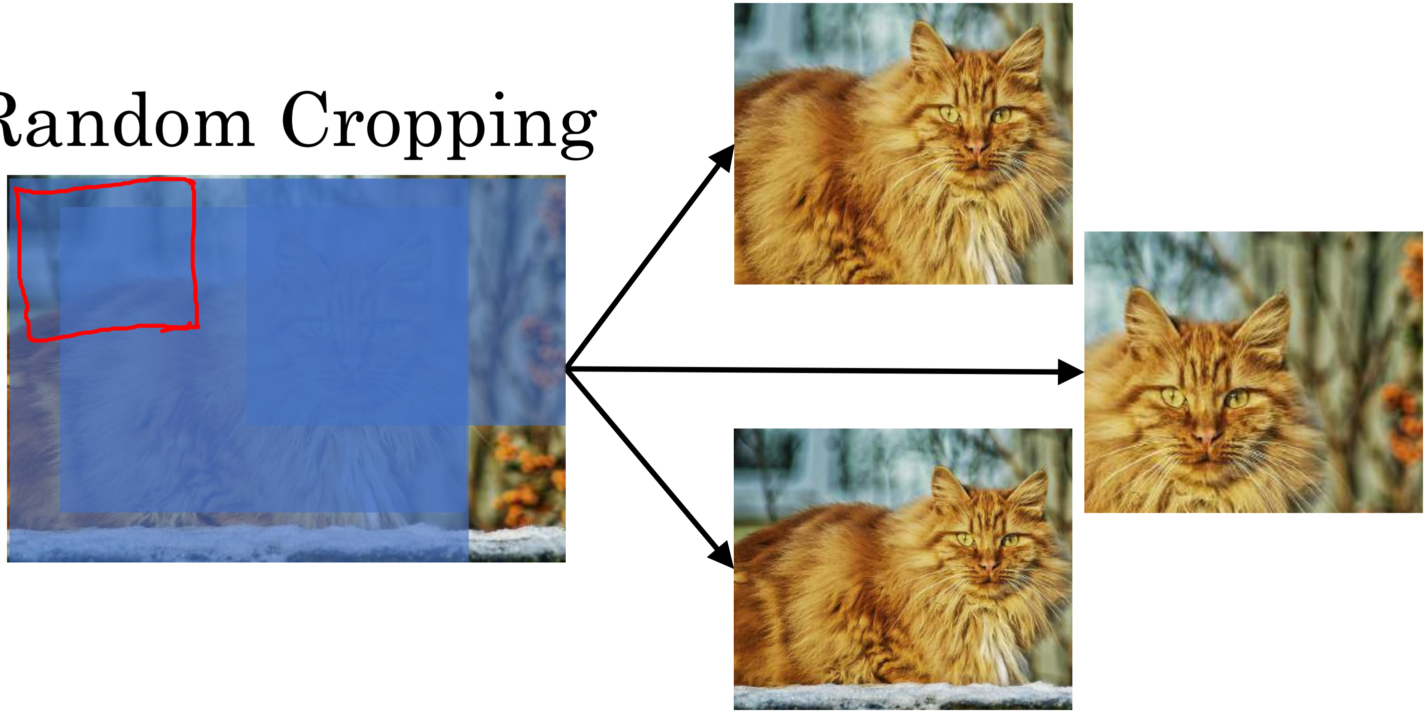
Common augmentation method

Mirroring



y

Random Cropping



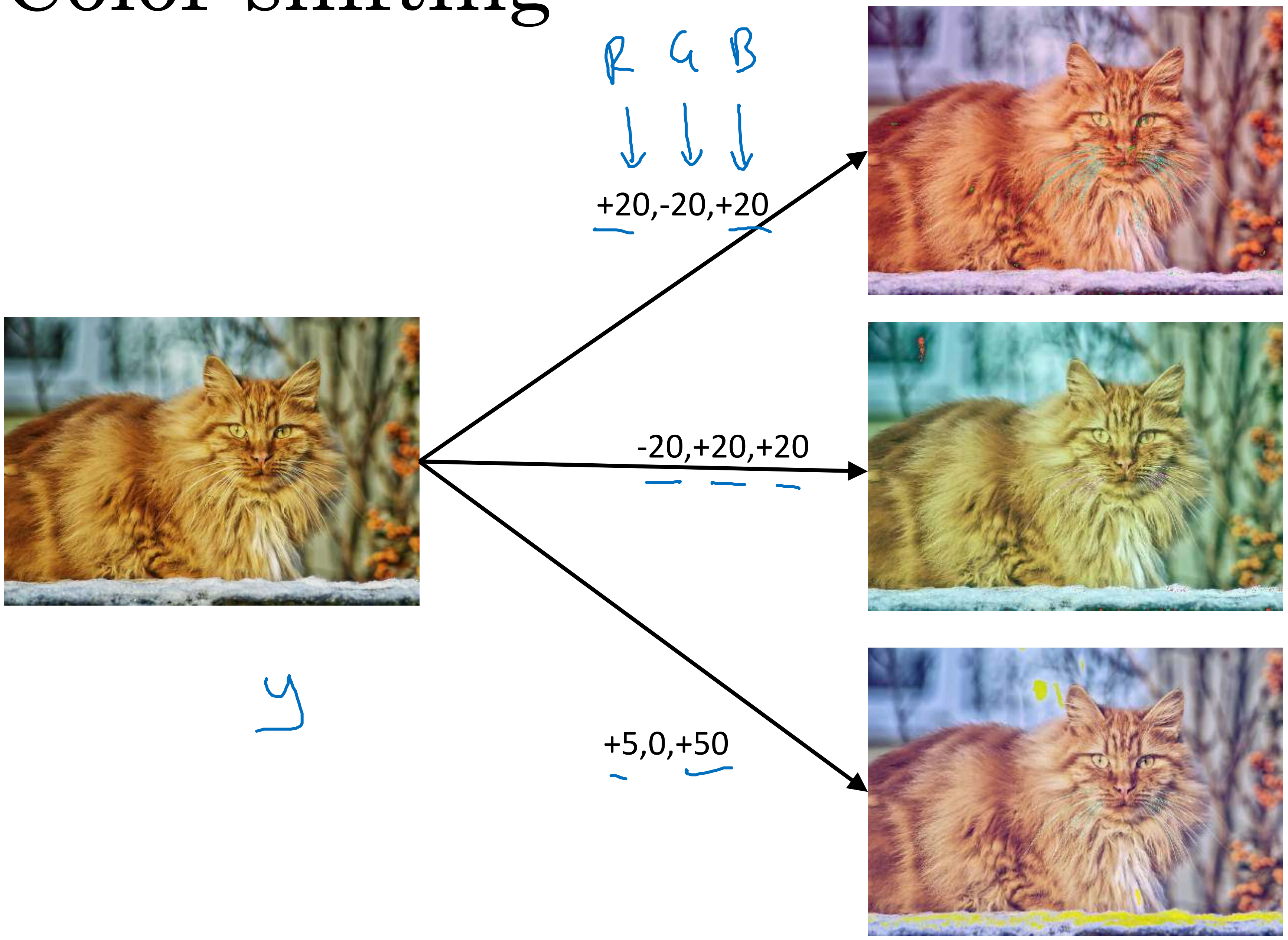
Rotation

Shearing

Local warping

...

Color shifting



Advanced:

PCA

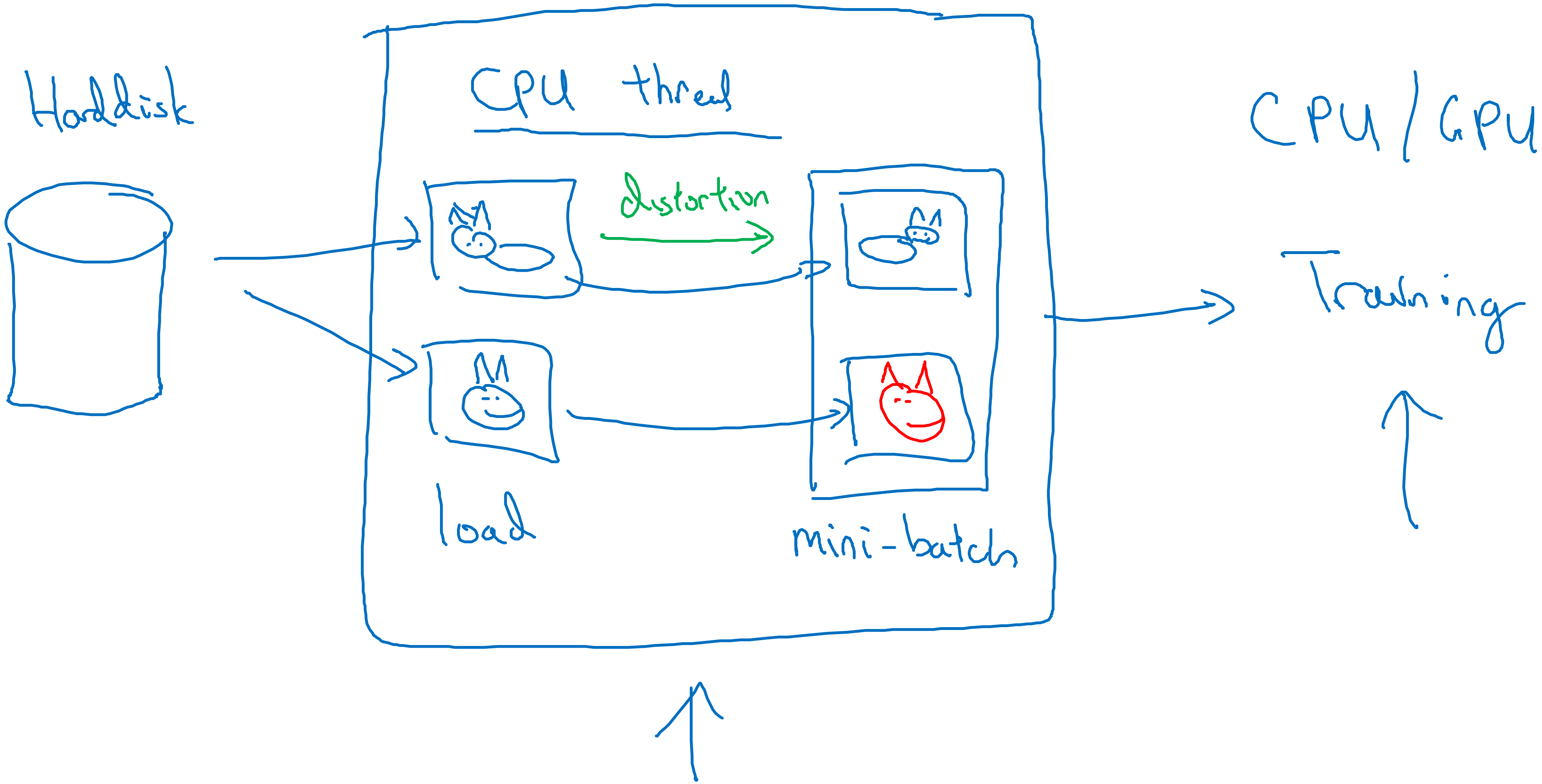
ml-class.org

AlexNet paper

"PCA color augmentation."

R B G

Implementing distortions during training



deeplearning.ai

Practical advice for using ConvNets

The state of computer vision

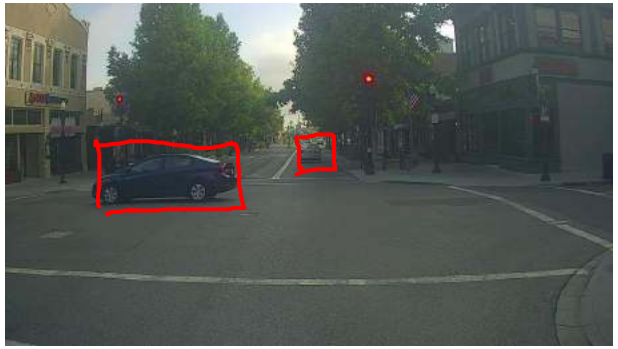
Data vs. hand-engineering

Little data | Lots of data

More hand-engineering ("hacks") | Object detection | Image recognition | Speech recognition | Simpler algorithms less hand-engineering

Tigger / Misty / neither.

Transfer learning



Two sources of knowledge

- Labeled data (x, y)
- Hand engineered features/network architecture/other components

Andrew Ng

Tips for doing well on benchmarks/winning competitions

Ensembling

3-15 networks → $\hat{y}$

- Train several networks independently and average their outputs

Multi-crop at test time

- Run classifier on multiple versions of test images and average results

10-crop

1 + 4 + 1 + 4

Use open source code

- Use architectures of networks published in the literature
- Use open source implementations if possible
- Use pretrained models and fine-tune on your dataset